Eddie May Ncube

Employee ID : ZIDIOkNAyE8

Project: RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform

1. Project overview

RetailPulse turns the Online Retail II transactions into sales insights, customer segments, a churn score, a 30-day revenue forecast and an inventory plan. The results are delivered as a live Streamlit app and a four-page Power BI report.

The project was built for the Zidio internship, which offers two tracks: data analysis with dashboards, and a Streamlit application. I completed both. A single command, `run_pipeline.py`, cleans the data, builds the features, trains the models and writes the files that both dashboards read.

The goals were to:

- Understand when, where and what customers buy.
- Segment customers by value and flag those likely to stop buying.
- Forecast daily revenue for the next 30 days and measure how accurate the forecast is.
- Recommend safety stock, reorder points and order quantities for the top 50 products.
- Publish the work as a public, working application with its source code on GitHub.

2. Dataset, custom row and cleaning

The data is the public Online Retail II dataset (UCI / Kaggle): transactions from a UK online gift retailer. I used the 2010–2011 sheet, which holds roughly half a million-line items, each with an invoice, product code, description, quantity, price, timestamp, customer ID, and country.

The assignment said not to use the data as downloaded, so I added one custom record and flagged it with is_custom_row, which lets it be traced through every stage.

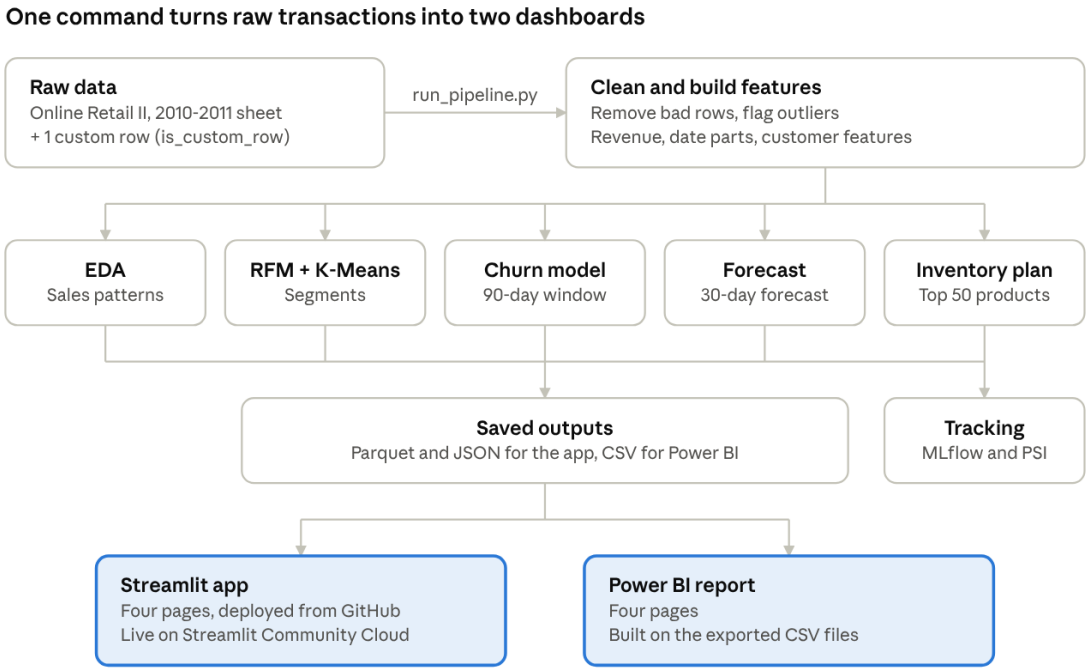
Field	Value
Invoice	999001
Stock code	RP-0001
Description	RETAILPULSE CUSTOM GIFT SET
Quantity	12
Unit price	£4.95
Invoice date	2011-12-09 12:30
Customer ID	99999
Country	United Kingdom

1. **Cleaning:** (src/cleaning.py) removes exact duplicates, rows with quantity or price of zero or below, non-product codes (postage, fees, samples), and rows with no description. Cancelled invoices are kept as returns rather than deleted. Extreme order values are flagged as outliers and excluded from the dashboards, not dropped from the data. A cleaning report is saved as JSON on every run.
2. **Feature engineering**: (src/features.py) adds revenue, date, year-month, day of week, hour, and a has-customer flag for guest checkouts. At the customer level, it builds recency, frequency, monetary value, tenure, average order value, return rate, and 90-day activity.
3. **Technology stack**: Everything runs in Python 3.12 except the Power BI report. Airflow and Docker files exist in the repository but were not run or built for this submission.

Area	Tools	Used for
Data handling	pandas, NumPy, PyArrow	Loading, cleaning, feature tables, Parquet outputs
Segmentation	scikit-learn (K-Means)	RFM scoring and customer clusters, with k chosen by silhouette score
Forecasting	Prophet, TensorFlow (LSTM), scikit-learn	Seasonal baseline, gradient boosting, Prophet, LSTM and a hybrid average
Churn	scikit-learn, Optuna	Logistic regression, random forest, gradient boosting and a tuned model
Experiment tracking	MLflow (SQLite)	Logging parameters and metrics for each model run
Monitoring	Custom PSI check	Population stability index to detect data drift
App	Streamlit, Plotly	Four-page interactive dashboard
BI	Power BI Desktop	Four-page report built on exported CSV files
Quality	pytest	Five automated tests of the pipeline
Delivery	GitHub, Streamlit Community Cloud	Source control and the public deployment
Written but not run	Airflow DAG, Dockerfile	Scheduling and containerising the pipeline

4. Architecture

One pipeline cleans the data once, fans out into five analyses, saves their results as files, and feeds both dashboards from those same files.



pipeline architecture · 5 analyses, 2 dashboards

The command `run_pipeline.py` runs every stage in order. MLflow logs each model run, a population stability check watches for data drift, and the Streamlit app and the Power BI report read the same saved outputs.

5. Results

The forecast is the strongest result, the churn model is moderate, and the inventory plan rests on a stated assumption. Figures in square brackets are placeholders to copy from your own outputs.

Sales. Revenue is £10,001,626 across 19,772 orders and 4,334 identified customers. It rises through the autumn into the Christmas peak, the United Kingdom dominates by revenue, and most buying happens on weekdays around

midday. Orders without a customer ID (guest checkouts) are counted in revenue but not in customer counts.

Customers. RFM scoring and K-Means place 4,334 customers into segments such as Champions, Loyal Customers, At Risk, Cannot Lose Them and Hibernating, with an average spend of £1,959 per customer. The churn model, defined with a 90-day window and a cut-off date set 90 days before the end of the data, flags 393 customers as high risk, and the average churn probability across all customers is 0.40. Four models were compared on 3,367 training customers, 43% of whom churned. The best, a gradient boosting model tuned with Optuna, reached a ROC-AUC of 0.75, ahead of random forest (0.74) and logistic regression (0.73). That is a useful but moderate result. The strongest driver is purchase frequency, at 45% of feature importance, and because recency is tightly tied to how churn is defined, its importance should be read with care.

Demand forecast. Five approaches were compared on a 30-day hold-out, scored by WAPE (weighted absolute percentage error, lower is better).

Model	WAPE	MAE (£/day)	RMSE (£/day)
Gradient Boosting (best)	22.3%	10,821	17,021
Seasonal Naive (baseline)	23.1%	11,174	14,750
Hybrid average	22.8%	11,065	15,672
Prophet	27.2%	13,181	18,265
LSTM	27.7%	13,406	16,372

Gradient Boosting forecasts about £986K of revenue for the next 30 days, an average of £32.9K per day, with a band of ±1.65 residual standard deviations. It beats the seasonal baseline only slightly. With a single year of daily data, the weekly pattern is the main thing any model can learn.

Inventory. For the 50 highest-revenue products, the plan calculates safety stock (1.65 × demand standard deviation × √7-day lead time), a reorder point and an economic order quantity. Total recommended safety stock is about 18.5K units. Of

the 50 products, 21 are flagged "Reorder now". Stock on hand is simulated, as explained in the limitations below.

6. Dashboards and deployment

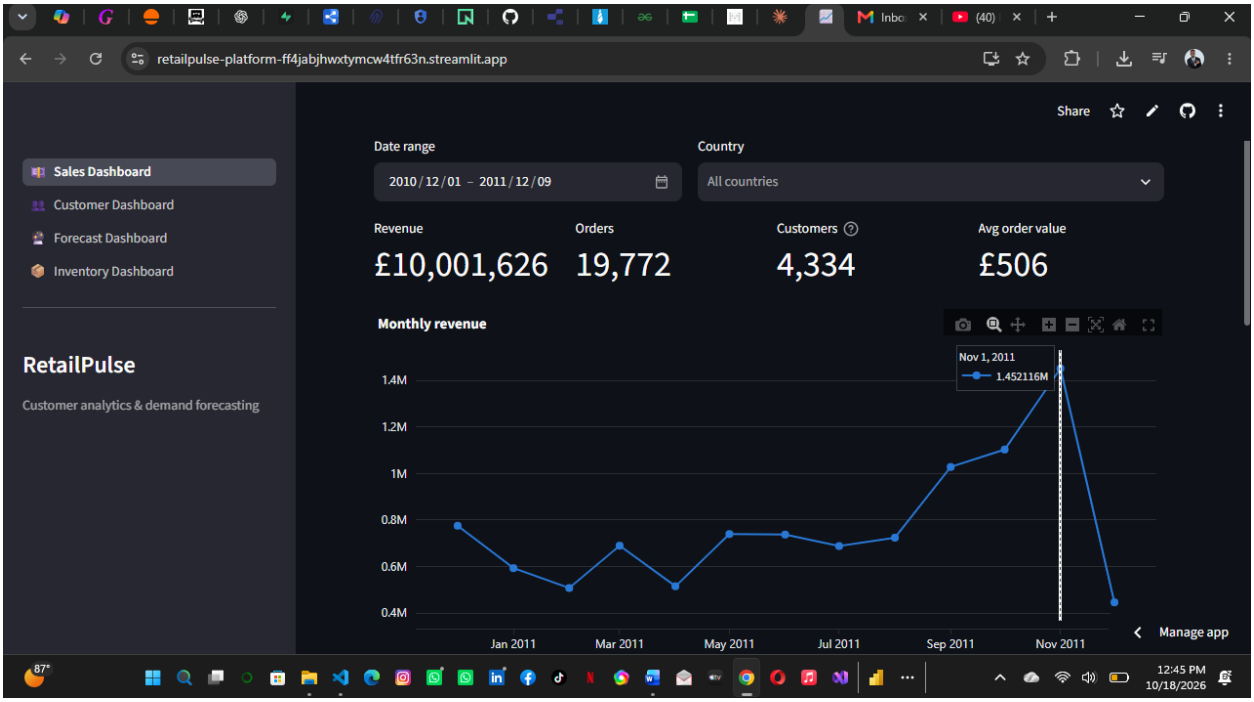
The same outputs feed two dashboards: a public Streamlit app and a Power BI report. Both have four pages.

Page	Streamlit app	Power BI report
Sales	Date and country filters, revenue KPIs, monthly trend, top products and countries, day-of-week and hour patterns	Revenue cards, monthly trend, country and product charts, slicers
Customers	Three tabs: RFM segments, churn risk with its top drivers and a high-risk watchlist, and a customer lookup	Customer and high-risk counts, segment donut, churn-risk columns, top at-risk table
Forecast	History, hold-out test and 30-day forecast with uncertainty band, model comparison	Actual versus forecast line chart, 30-day forecast cards, model accuracy note
Inventory	Reorder point chart, products by stock status and the full inventory plan table	Status donut, reorder table, demand chart, assumptions note

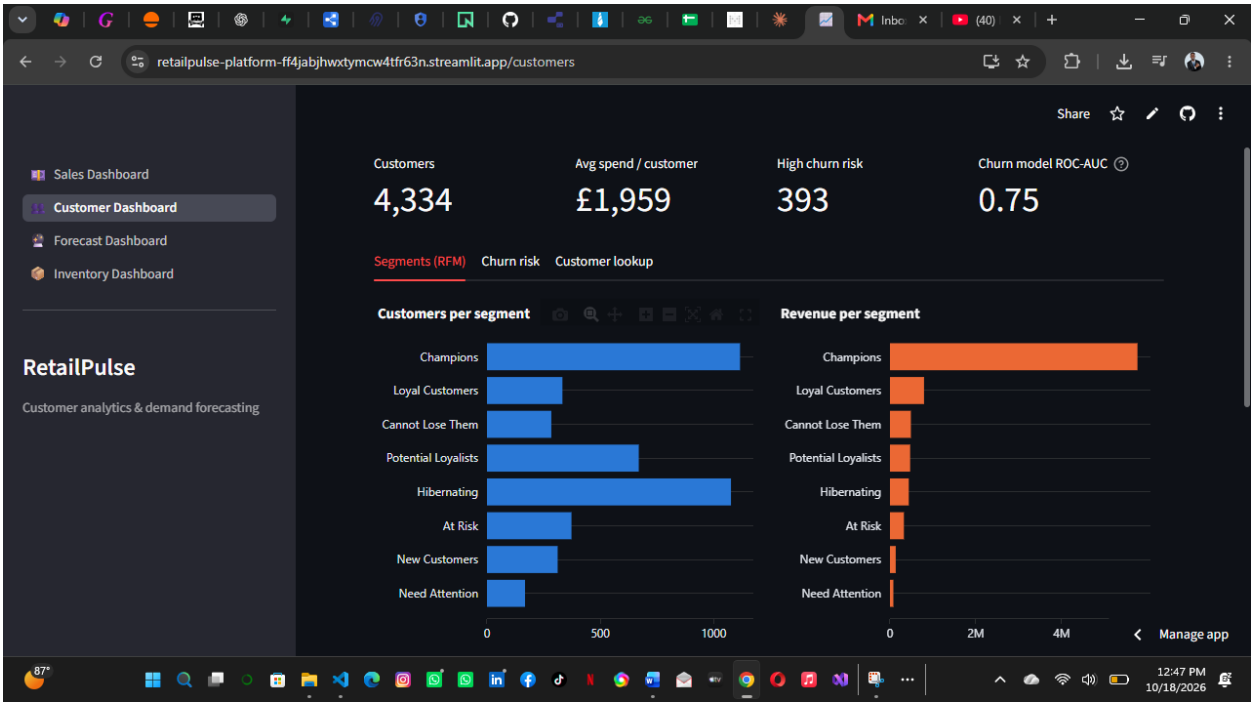
Deployment. The code is on GitHub and the Streamlit app runs on Streamlit Community Cloud. The app reads a lighter dependency file (`app/requirements.txt`) so the cloud build does not install TensorFlow or Prophet, and the sales page loads only the columns it needs to stay inside the free memory limit.

Screenshots

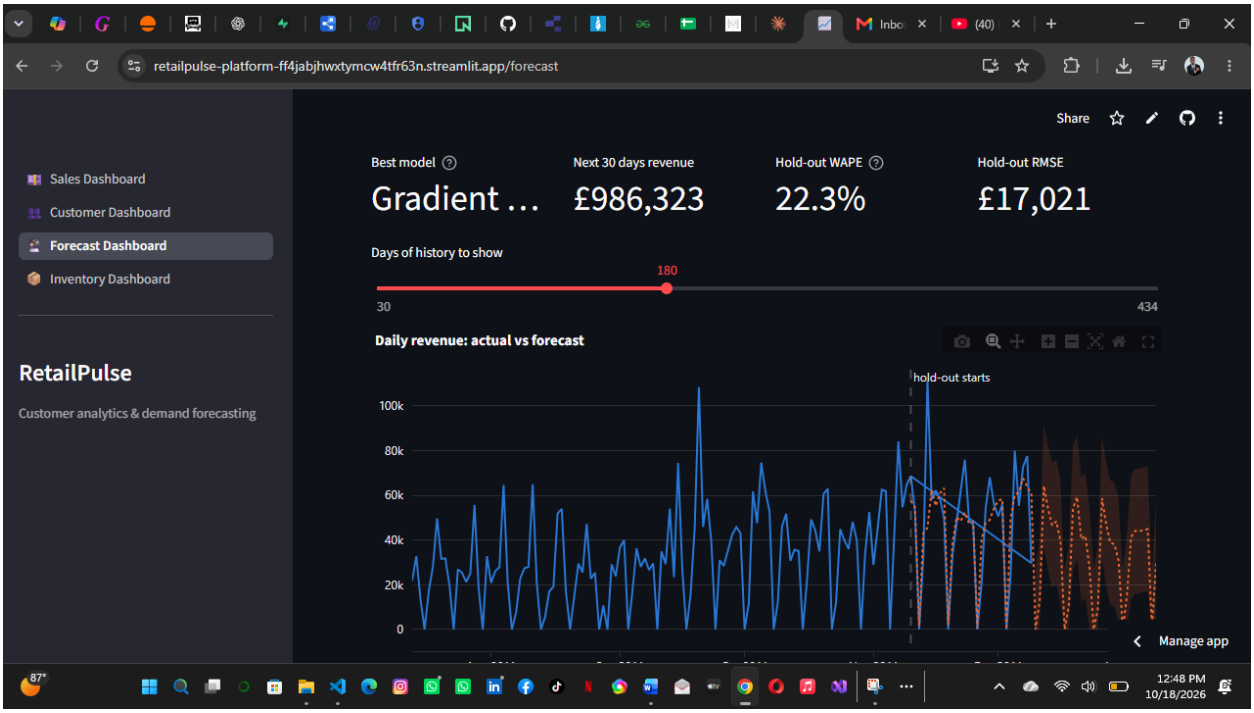
Streamlit – Sales dashboard



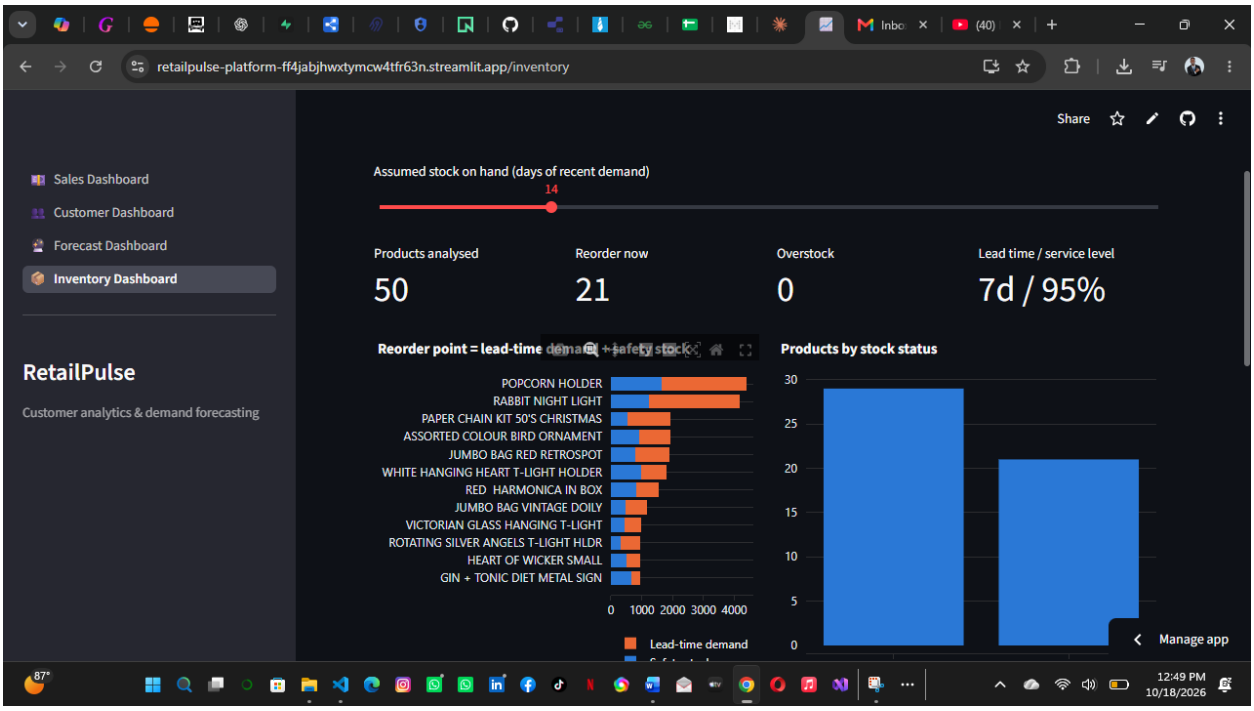
Streamlit – Customers dashboard



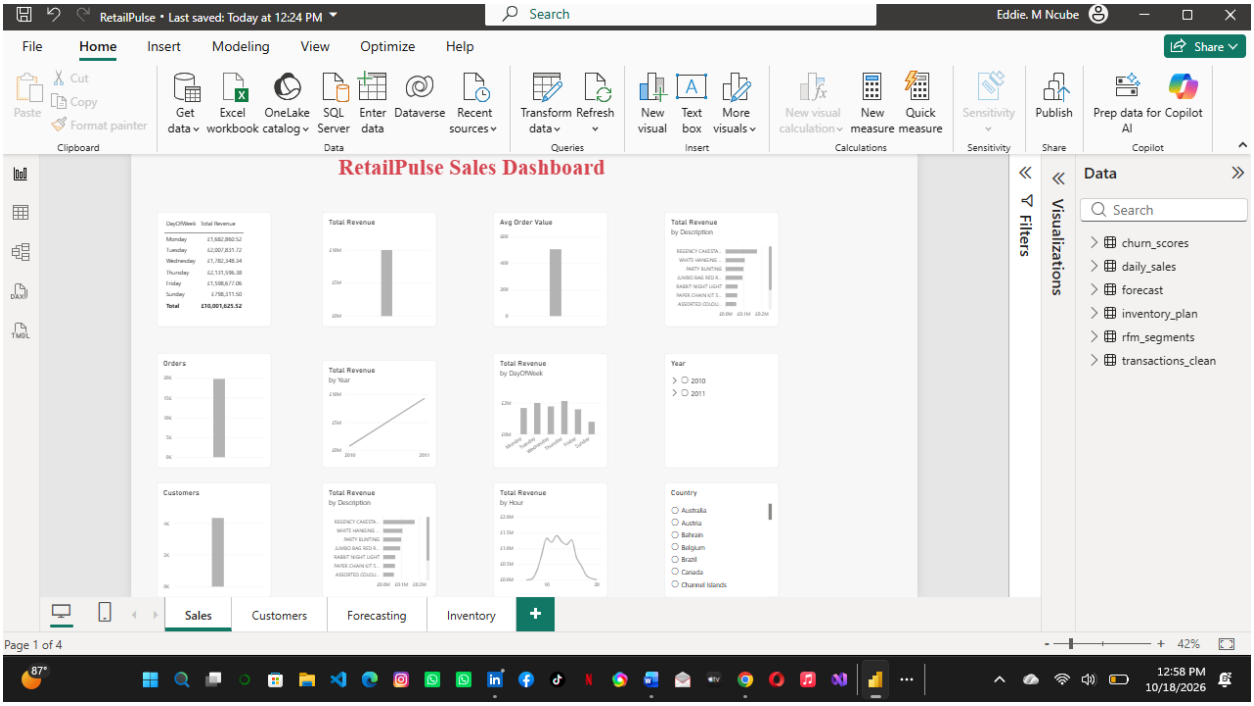
Streamlit – Forecast dashboard



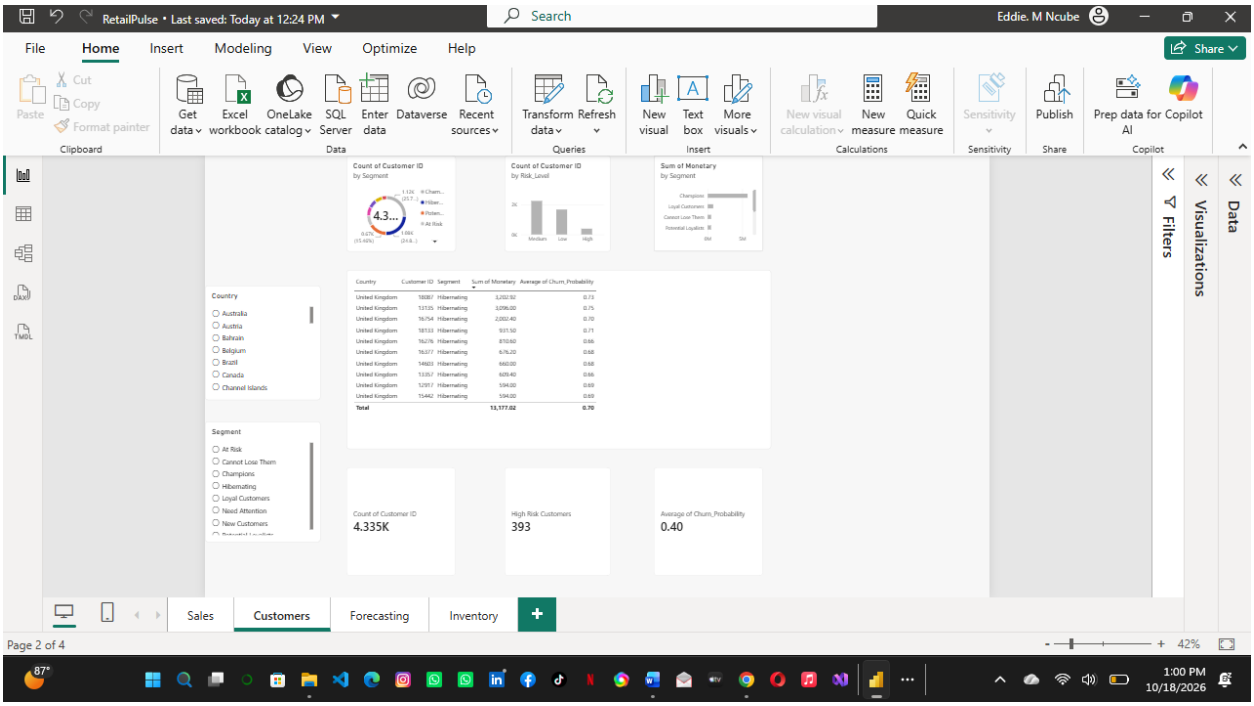
Streamlit – Inventory dashboard



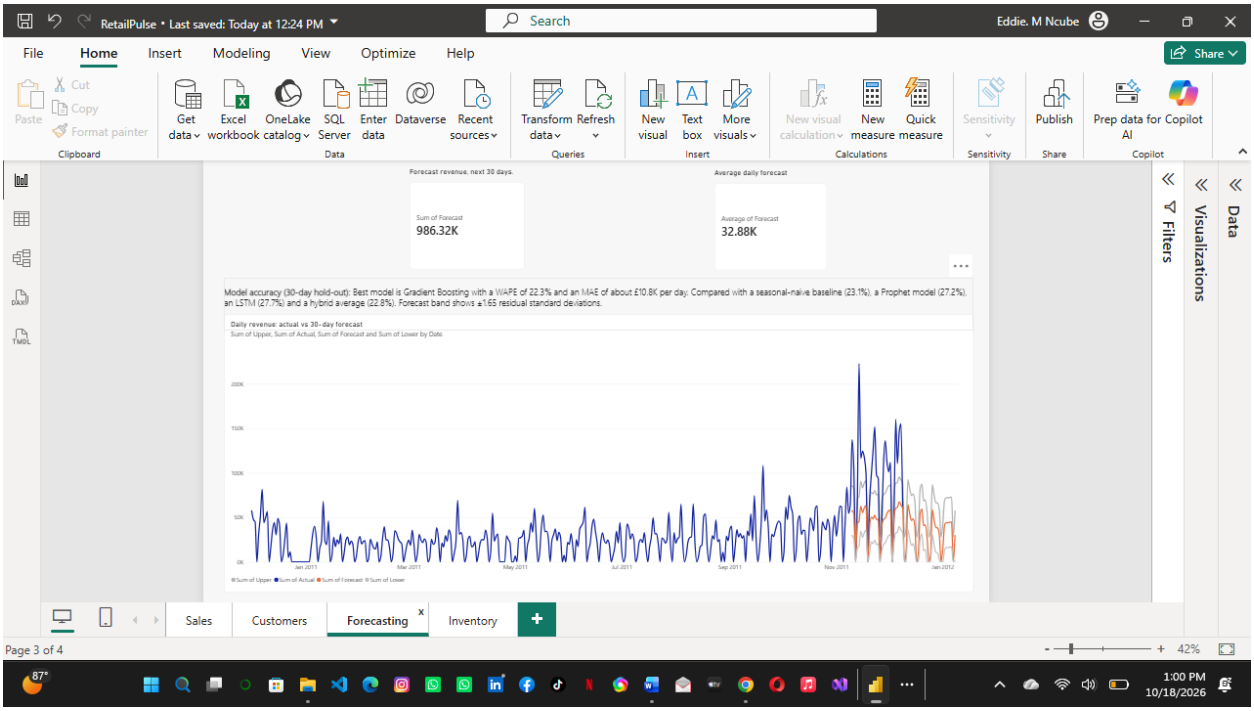
Power BI – Sales dashboard



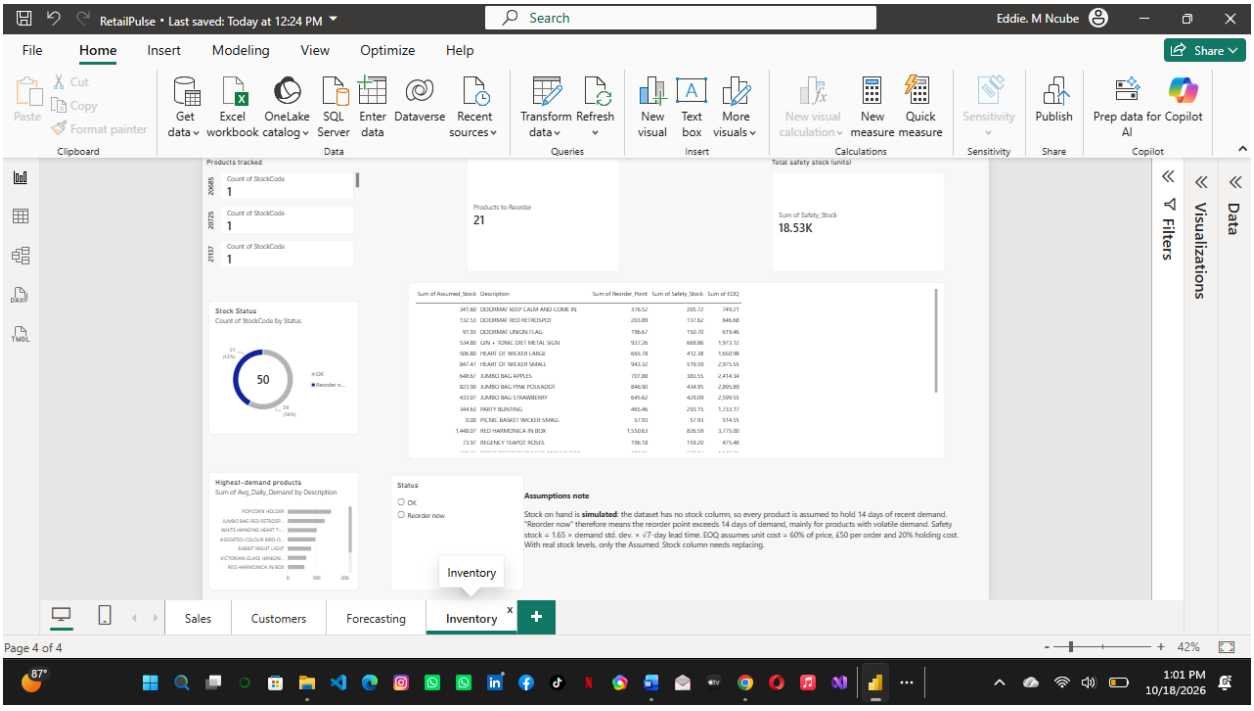
Power BI – Customers dashboard



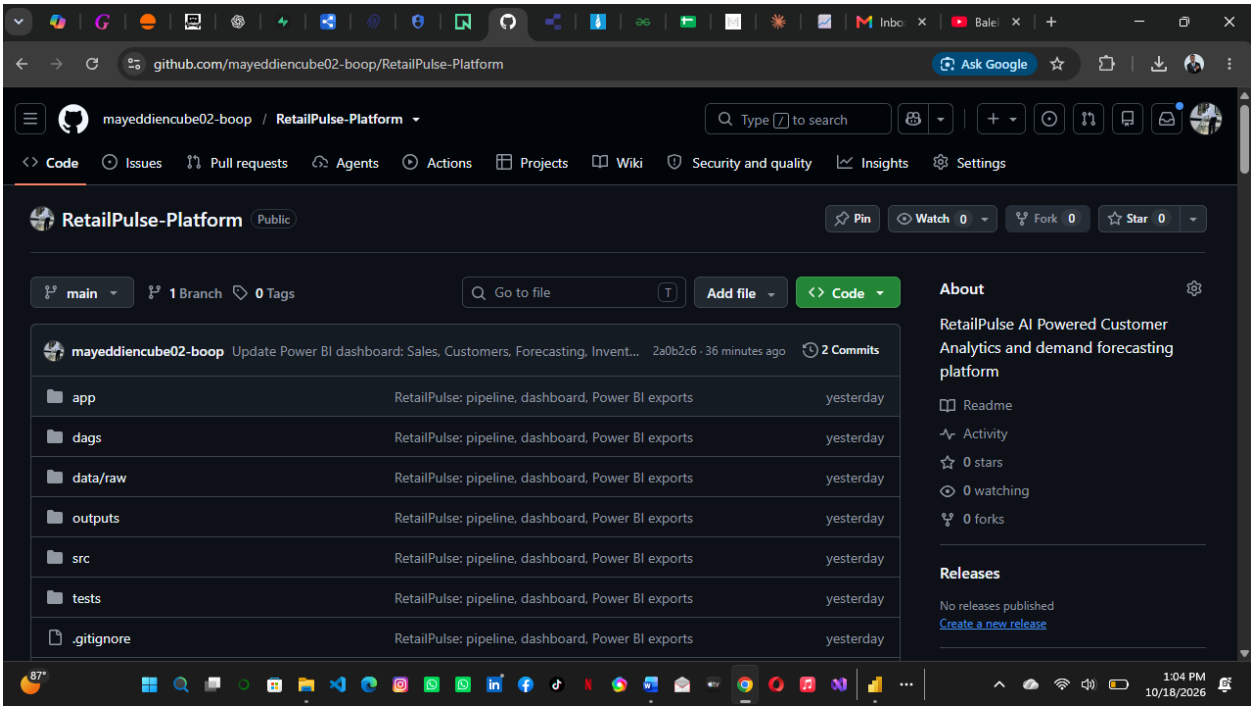
Power BI – Forecasting dashboard



Power BI – Inventory page



GitHub repository main page



7. Challenges and limitations

The biggest limitation is that the dataset has no stock-on-hand column, so the inventory status is illustrative, not real.

Limitations

- **Simulated stock.** Every product is assumed to hold 14 days of recent demand. "Reorder now" therefore means the reorder point is above 14 days of demand, mainly for products with volatile demand. Safety stock, reorder point and EOQ are genuine calculations. EOQ also assumes a unit cost of 60% of price, £50 per order and a 20% holding cost.
- **One year of data.** The forecast cannot learn yearly seasonality beyond a single Christmas peak, which is why the advanced models barely beat a seasonal baseline.
- **Churn definition.** Churn is defined as no purchase in a 90-day window, which suits this retailer's buying pattern but is a modelling choice.
- **Not run end to end:** the Airflow DAG and the Dockerfile are written but were not executed or built.

Challenges and how I solved them

Problem	Fix
TensorFlow would not install on the system Python 3.14	Rebuilt the virtual environment on Python 3.12
Reading the Excel file ran out of memory	The loader now parses only the sheet it needs
Power BI refused relationships because of blank customer IDs	Used the customer table that already carries segment and risk, and removed the relationships
Power BI read the forecast columns as	Converted them to decimal numbers

Problem	Fix
text because of blank cells	in Power Query
Forecast cards counted the test period and the future together	Filtered the cards to future rows only
The Sales page loaded about half a million rows, which risked the free tier's memory limit	Used a lighter dependency file and loaded only the required columns

8. Conclusion and future work

RetailPulse delivers a tested, end-to-end analytics pipeline with a live Streamlit app and a Power BI report, and it is honest about where its results are weak. The 30-day forecast is accurate to about 22% of daily revenue, 393 customers are flagged as churn risks, and the inventory plan gives reorder points and order quantities for the top 50 products.

What I learned is that the data limits the models more than the algorithms do: a simple seasonal baseline came close to every advanced model, and missing stock data meant the inventory status had to be simulated.

Next steps:

- Replace the simulated stock with real stock levels, so "Reorder now" reflects actual inventory.
- Train on several years of data to capture yearly seasonality.
- Schedule the pipeline with Airflow and containerise it with Docker, which are written but untested.
- Add automatic drift alerts and retraining when the population stability index rises.

9. Links

Deliverable	Link
Source code (GitHub)	mayeddiencube02-boop/RetailPulse-Platform
Live deployment (Streamlit)	retailpulse-platform app
Demo video	https://drive.google.com/file/d/1PxNxvLAGEmt8h6CaJU7KFTqUrWF9vEA5/view?usp=sharing
Feedback video	https://drive.google.com/file/d/1AD4fGuuv-CHZxY6O7E5Y1ErSbKRrriUa/view?usp=sharing

Dataset: Online Retail II (UCI) on Kaggle.